

Engram: Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models

Le-Van Thai

January 2026

1 Giới thiệu

Khi nghe câu "Alexander the Great." Khi nghe Alexander là mình biết ngay là nhân vật lịch sử La Mã mà không cần phải "reconstruct Alexanders identity" trong đầu hay kiểu suy nghĩ về các chi tiết tiểu sử của ông đó mà mình chỉ đơn giản là biết được ông đó là ai.

Não của mình thực hiện $O(1)$ lookup, retrieves the memory và tự nhiên mình hiểu được câu đó.

Nhưng với GPT, nó phải đi qua hơn 50 layers attention, tính toán các mối quan hệ, mà tất cả để mô phỏng một thao tác tra cứu bảng băm đơn giản. Nó giống việc dùng supercomputer để tìm số điện thoại của ai đó thay vì tìm số điện thoại từ phonebook.

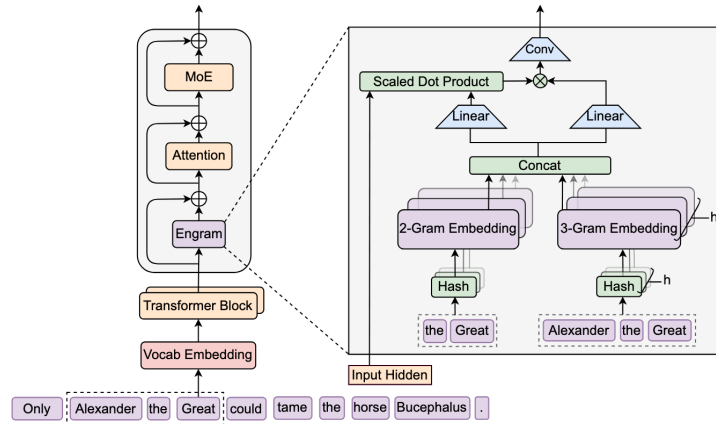
Sẽ có những phần khó, cần suy luận nhiều để hiểu được ngữ cảnh xuyên suốt hoặc muốn biết 'It' là chỉ cái gì thì phải đọc lại những câu trước đó $\rightarrow$ Những việc này cần attention, độ sâu, và tính toán thực sự.

Tuy nhiên, những cụm từ cơ bản hoặc tên những người ai cũng biết như Bác Hồ, nhân tiện, phép toán cộng trừ. Chúng không cần lập luận. Chúng chỉ cần bộ nhớ. Nhưng transformer thì không có bộ nhớ, nên chúng lãng phí độ sâu quý giá để tái dựng những khuôn mẫu này từ đầu mỗi lần.

2 The Engram Architecture

DeepSeek đưa ra giải pháp là add một lookup table và họ đặt tên là Engram - là một thuật ngữ khoa học thần kinh dùng để chỉ dấu vết trí nhớ. Tuy nhiên để hiểu thì có thể xem nó như 1 cuốn sổ tay thông minh trong trong Transformer.

Kiến trúc cho thấy Engram hoạt động song song với cơ chế attention, thay vì buộc toàn bộ thông tin phải đi qua attention. Khi mô hình xử lý một thực thể đã quen thuộc (ví dụ "Alexander the Great"), Engram sẽ chủ động truy hồi ký ức liên quan, trong khi attention đảm nhiệm việc phân tích ngữ cảnh và quan hệ trong câu.



Module bộ nhớ chỉ được tích hợp vào một số tầng transformer được chọn nhằm decouple bộ nhớ khỏi phần tính toán, trong khi vẫn giữ nguyên tokenizer, input embedding và output projection của mô hình gốc.

Khi gặp thông tin quen (vd: “Alexander the Great”):

$$\text{output} \approx \text{Attention}(x) + \lambda \cdot \text{Memory}(x), \quad \lambda \text{ lớn.}$$

Khi mô hình gặp thông tin không quen:

$$\text{output} \approx \text{Attention}(x), \quad \lambda \approx 0.$$

2.1 Stage 1: Lookup

Trước hết, họ nén vocab của tokenizer lại 23%. Lý do là vì các standard tokenizers không tốt vì: chúng coi "Apple", "Apple" và "apple" là những token hoàn toàn khác nhau, khiến các từ có cùng ý nghĩa bị rải rác khắp không gian ID. Phép gộp lớn nhất là whitespace, khi 163 token khác nhau (tab, xuống dòng, khoảng trắng) được gom lại thành một. Chữ cái 'a' cũng được gộp từ 54 biến thể khác nhau, bao gồm 'A', 'á', 'ä', 'à'. Quá trình chuẩn hoá NFKC kết hợp với case-folding này tạo ra một ánh xạ surjective từ các token ID thô sang các định danh chuẩn, qua đó tối đa hoá mật độ ngữ nghĩa.

```
self.normalizer = normalizers.Sequence([
    normalizers.NFKC(),
    normalizers.NFD(),
    normalizers.StripAccents(),
    normalizers.Lowercase(),
    normalizers.Replace(Regex(r"[\t\r\n]+"), "_"),
    normalizers.Replace(Regex(r"^[_]$"), SENTINEL),
    normalizers.Strip(),
    normalizers.Replace(SENTINEL, "_"),
])
```

Tại mỗi vị trí đang xử lý, Engram nhìn vào một đoạn ngắn các từ ngay trước đó (local context), rồi lấy các cụm từ ở cuối đoạn này. Ví dụ khi đang xử lý “Great”, Engram lấy các cụm như “the Great” và “Alexander the Great”. Mặc định, hệ thống dùng các cụm 2 từ và 3 từ (2-gram và 3-gram).

Multi-head hashing cho N-gram embedding. Thay vì lưu embedding cho mọi N-gram vào cùng một bảng (điều không khả thi), Engram sử dụng *multi-head hashing* để biểu diễn N-gram.

Vì sao không thể lưu embedding cho mọi N-gram? Giả sử vocabulary sau khi chuẩn hóa có kích thước 50 000 token và mô hình sử dụng 2-gram và 3-gram. Khi đó, số lượng N-gram khả dĩ là:

$$\#2\text{-gram} = 50\,000^2 \approx 2.5 \times 10^9,$$

$$\#3\text{-gram} = 50\,000^3 \approx 1.25 \times 10^{14}.$$

Nếu mỗi N-gram được gán một embedding có 1280 chiều và sử dụng `float16`, thì chỉ riêng các 2-gram đã cần khoảng 6.4 TB bộ nhớ, trong khi các 3-gram hoàn toàn vượt quá khả năng lưu trữ và huấn luyện thực tế. Mặc dù trong thực tế không phải mọi N-gram đều xuất hiện, nhưng trong bối cảnh văn bản open-domain (tên riêng mới, thực thể mới), không thể biết trước N-gram nào sẽ gặp. Do đó, không thể tiền liệt kê hay huấn luyện embedding theo cách của tokenizer truyền thống. Hiện tượng này được gọi là *combinatorial explosion*.

Giải pháp: Multi-head hashing. Thay vì lưu embedding theo từng N-gram, Engram lưu embedding theo *hash bucket*. Mỗi N-gram được băm qua K hàm hash độc lập (ví dụ $K = 8$), trong đó mỗi hàm hash trỏ tới một bảng embedding riêng có kích thước cố định (khoảng 50 000, thường chọn là số nguyên tố). Tổng số embedding cần lưu khi đó chỉ là:

$$8 \times 50\,000 = 400\,000,$$

nhỏ hơn rất nhiều so với hàng tỷ hoặc hàng nghìn tỷ N-gram khả dĩ.

Xử lý khi gặp một N-gram mới. Với một N-gram chưa từng xuất hiện (ví dụ “Alexander the Great”), hệ thống không tạo embedding mới. Thay vào đó, chuỗi này được đưa qua 8 hàm hash để xác định 8 bucket khác nhau; từ mỗi bucket, một vector embedding có sẵn được lấy ra. Các vector này được nối (concatenate) lại để tạo thành embedding 1280 chiều cho N-gram. Nhờ sử dụng nhiều hash head, xác suất hai N-gram khác nhau va chạm hoàn toàn là rất thấp. Do đó, mọi N-gram đều có biểu diễn ngay lập tức, không phụ thuộc vào việc đã từng xuất hiện hay chưa, đồng thời tránh được bùng nổ tổ hợp.

2.2 Stage two: Trust But Verify

Tra cứu băm thô thường gây nhiễu. Xung đột băm xảy ra do từ ngữ có nhiều nghĩa (đa nghĩa). Vì vậy, Engram bổ sung thêm cơ chế lọc theo ngữ cảnh.

Xem qua Hình 2, sau bước nối (concatenation) hai embedding N-gram. Trạng thái ẩn hiện tại được xem như *Query*, trong khi embedding N-gram sau khi được đưa qua phép chiếu tuyến tính (linear projection) đóng vai trò là *Key* và *Value*.

Mô hình tính toán một hệ số cổng (gating scalar) $\alpha \in [0, 1]$ nhằm quyết định mức độ tin cậy của thông tin được truy xuất từ bộ nhớ. Trước khi tính tích vô hướng có chuẩn hoá (scaled dot-product) và đưa qua hàm sigmoid, *RMSNORM* được áp dụng cho cả *Query* và *Key*.

Khi α tiến gần 1, thông tin truy xuất từ bộ nhớ được ưu tiên sử dụng; ngược lại, khi α tiến gần 0, mô hình giảm ảnh hưởng của bộ nhớ và chủ yếu dựa vào quá trình tính toán nội tại.

Đây là một tổ hợp (mixture) các embedding, trong đó router cũng là các embedding. MoE truyền thống là tổ hợp các FFN, trong đó router cũng là FFN nhưng sử dụng mạng khóa (key network) chỉ có bias. Engram tuân theo cùng một khuôn mẫu toán học, nhưng áp dụng nó lên bộ nhớ (memory) thay vì khối tính toán (computation).

Mô hình sử dụng một phép depthwise causal convolution ngắn, với kích thước kernel là 4 và dilation được đặt bằng bậc N-gram tối đa, kết hợp với SiLU nhằm mở rộng vùng tiếp nhận cục bộ (receptive field) và tăng cường tính phi tuyến.

Đầu ra của phép tích chập này được cộng vào luồng dư (residual stream) của Transformer như một tín hiệu ngữ cảnh cục bộ bổ sung, cho phép cơ chế attention tự động chọn lọc hoặc bỏ qua thông tin tùy theo mức độ hữu ích.

$$\text{output} \approx \text{Attention}(x) + \lambda \cdot \text{Memory}(x), \quad \lambda \text{ lớn.}$$

Khi mô hình gặp thông tin không quen:

$$\text{output} \approx \text{Attention}(x), \quad \lambda \approx 0.$$

Kết quả ablation cho thấy khi bỏ cơ chế gating theo ngữ cảnh, hiệu năng của mô hình giảm mạnh. Việc loại bỏ nén tokenizer và tích hợp đa nhánh cũng dẫn đến suy giảm tương tự. Những thành phần này không mang tính bổ sung cho đẹp, mà đóng vai trò cốt lõi trong kiến trúc.

Trong MoE, quyết định routing chỉ được xác định trong lúc forward pass, dẫn đến khó khăn về scheduling và làm giảm khả năng song song hóa. Cơ chế truy xuất của Engram mang tính xác định (deterministic) và chỉ phụ thuộc vào các token đầu vào. Do đó, có thể biết chính xác những embedding nào cần được truy xuất ngay cả trước khi bắt đầu quá trình tính toán. Các chỉ mục truy xuất (retrieval indices) có thể được xác định chỉ từ chuỗi đầu vào, không cần đến các trạng thái ẩn được tính toán trong lúc chạy.

During training, các bảng embedding được phân mảnh (shard) trên nhiều GPU bằng standard model parallelism. An All-to-All communication được sử

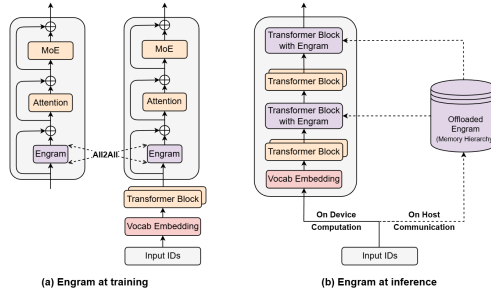


Figure 2 | **System implementation of Engram.** (a) Training Phase: The massive embedding tables are sharded across available GPUs. An All-to-All communication primitive is employed to retrieve active embedding rows across devices. (b) Inference Phase: Engram tables are offloaded to host memory. By exploiting the deterministic retrieval logic, the host asynchronously prefetches and transfers embeddings, overlapping communication with the on-device computation of preceding Transformer blocks.

dụng để thu thập các hàng embedding đang hoạt động trong forward pass và truyền gradient ngược lại trong backward pass, nhờ đó dung lượng bộ nhớ tổng thể tăng tuyến tính theo số accelerator.

Ví dụ:

Giả sử một bảng embedding lớn được chia (shard) trên 4 GPU theo song song mô hình:

$$\begin{aligned} \text{GPU}_0 &: \text{rows } [0, \dots, 249,999,999] \\ \text{GPU}_1 &: \text{rows } [250,000,000, \dots, 499,999,999] \\ \text{GPU}_2 &: \text{rows } [500,000,000, \dots, 749,999,999] \\ \text{GPU}_3 &: \text{rows } [750,000,000, \dots, 999,999,999]. \end{aligned}$$

Với một chuỗi đầu vào, mô hình cần truy xuất các embedding có chỉ số:

$$\mathcal{I} = \{10, 42, 300\text{M}, 800\text{M}\}.$$

Trong forward pass, mỗi GPU chỉ truy xuất các embedding thuộc shard cục bộ của mình:

$$\begin{aligned} \text{GPU}_0 &: \{10, 42\}, \\ \text{GPU}_1 &: \{300\text{M}\}, \\ \text{GPU}_3 &: \{800\text{M}\}. \end{aligned}$$

Sau đó, một phép giao tiếp *All-to-All* được sử dụng để trao đổi các embedding đang hoạt động, nhằm đảm bảo mỗi GPU nhận được đầy đủ các embedding cần thiết cho quá trình tính toán tiếp theo.

Trong backward pass, gradient của mỗi embedding được gửi ngược về GPU sở hữu shard tương ứng:

$$\nabla e_{10}, \nabla e_{42} \rightarrow \text{GPU}_0, \quad \nabla e_{300\text{M}} \rightarrow \text{GPU}_1, \quad \nabla e_{800\text{M}} \rightarrow \text{GPU}_3.$$

Phép giao tiếp *All-to-All* đảm bảo các gradient được định tuyến đúng về shard gốc để cập nhật tham số.

Nhờ cơ chế phân mảnh embedding kết hợp với giao tiếp All-to-All trong forward và backward pass, dung lượng bộ nhớ tổng thể của mô hình tăng tuyến tính theo số accelerator.

Trong giai đoạn suy luận, cơ chế truy xuất của Engram là xác định trước vì các embedding cần dùng chỉ phụ thuộc vào chuỗi token đầu vào và không phụ thuộc vào các trạng thái ẩn được tính toán trong lúc chạy. Do đó, ngay trước khi GPU bắt đầu forward pass, mô hình đã biết chính xác những embedding nào sẽ cần được sử dụng. Thay vì lưu trữ sẵn các embedding này trong bộ nhớ HBM đắt đỏ trên GPU, chúng được giữ trong bộ nhớ host (DRAM) có dung lượng lớn và chi phí thấp hơn. Các embedding được lấy trước (prefetch) từ DRAM trong khi GPU đang xử lý các lớp khác của mô hình, nhờ đó quá trình truyền dữ liệu qua PCIe có thể chồng lấp với tính toán. Vì các embedding chỉ xuất hiện trên GPU đúng thời điểm cần thiết và không tồn tại thường trực, cơ chế này giúp loại bỏ nhu cầu lưu trữ bằng embedding lớn trong bộ nhớ HBM.

Bài báo đặt Engram tại các **layers 2 and 15**, và đây không phải là lựa chọn ngẫu nhiên. Layer 2 đủ sớm để chuyển giao việc tái cấu trúc các mẫu cục bộ trước khi backbone lãng phí độ sâu cho nhiệm vụ này, nhưng cũng đủ muộn để một vòng attention đầu tiên đã tạo ra các trạng thái ẩn được ngữ cảnh hóa một cách có ý nghĩa, từ đó cho phép cơ chế gating hoạt động chính xác. Layer 15 đóng vai trò là điểm chèn thứ hai, nơi ngữ cảnh đã trở nên phong phú hơn. Thiết kế theo tầng này giúp dung hòa đánh đổi giữa năng lực mô hình hóa và độ trễ hệ thống.

Các tác giả đã kiểm chứng điều này với một bảng Engram khổng lồ 100 tỷ tham số, được lưu hoàn toàn trong bộ nhớ host (DRAM). Mức suy giảm thông lượng là bao nhiêu? Dưới 3%. Cụ thể, với backbone dense 4B tham số, thông lượng đạt 9,031 token/giây ở baseline và 8,858 token/giây khi kết hợp với Engram 100B. Với backbone dense 8B, con số tương ứng là 6,315 so với 6,140 token/giây. Mức chênh lệch này là không đáng kể.

Đáng chú ý, các kết quả trên được đo trong một kịch bản cơ sở mang tính bảo thủ, trong đó mọi embedding đều được truy xuất trực tiếp từ bộ nhớ host và truyền qua PCIe, không sử dụng bất kỳ cơ chế cache nào trên GPU. Trong một hệ thống triển khai thực tế, nhờ tính cục bộ theo phân bố Zipf—tức là một số nhỏ các mẫu xuất hiện với tần suất rất cao—các embedding được truy xuất thường xuyên có thể được giữ sẵn trong bộ nhớ HBM trên GPU, trong khi các embedding hiếm hơn mới được lấy từ DRAM. Cách làm này giúp giảm đáng kể số lần truyền qua PCIe, và do đó hiệu năng thực tế còn có thể cao hơn các kết quả benchmark được báo cáo